CE2003 Tut 1

Q1 Draw the circuit described by the following Verilog module:

```
module arbitrary (input clk, a, b, input [2:0] c,
                  output y, output reg z);

   and g1 (w1, a, c[2]);
   not g2 (nc, c[0]);
   xor g3 (w2, c[1], b);
   nor g4 (pz, w1, w2, nc);
   and g5 (y, w2, nc);

   always @ (posedge clk)
      z <= pz;

endmodule
```

Q2 Draw a set of cascaded D-type flip-flops to implement a shift register and label the outputs of all flip-flops based on the following code fragment:

```
always @(posedge clk)

begin
   if (rst) begin
      c <= 1'b0; g <= 1'b0; i <= 1'b0; l <= 1'b0; o <= 1'b0;
   end else begin
      o <= l;
      c <= i;
      g <= o;
      l <= srin;
      i <= g;
   end
```

Q3 (a) Write a Verilog module that implements a 3-bit up/down counter. Now write a top level module that instantiates your counter and connects its output to an instance of the following circuit:

```
module checkit (input [2:0] numin, output divs)

   assign divs = (~numin[2]&~numin[1]&~numin[0]) |
                 (~numin[2]&numin[1]&numin[0]) |
                 (numin[2]&numin[1]&~numin[0]);

endmodule
```

(b) Draw a timing diagram showing how your counter output changes, assuming it is set to count down. Show the resulting output of the top level module.

(c) Hence, deduce the function of the code in (b).

Q4 Briefly explain the meaning of Functional Verification and Timing Verification.

Q1 Firstly, consider this code. What is wrong with the code?

```
module arbitrary (input clk, a, b, input [2:0] c, output [1:0] y);
    and g1 (w1, a, c[2]);
    not g2 (nc, c[0]);
    xor g3 (w2, c[1], b);
    nor g4 (py, w1, w2, nc);
    and g5 (y[0], w2, nc);

    always @ (posedge clk)
      y[1] <= py;

  endmodule
```

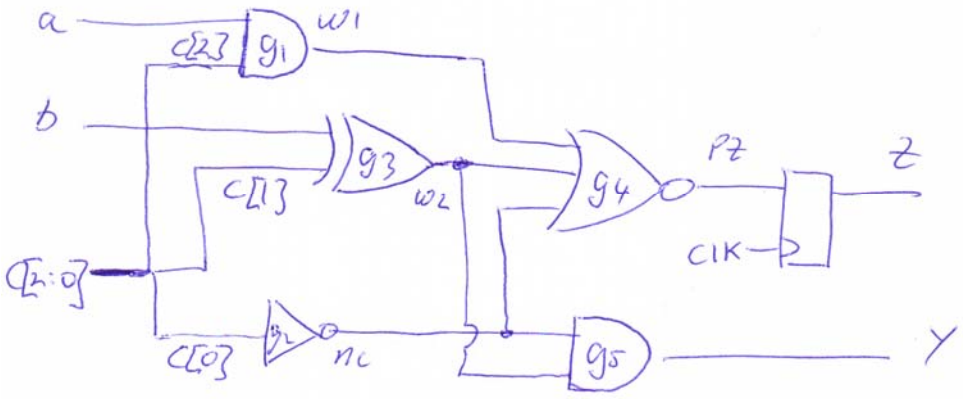
Now the actual tutorial question. What does it implement?

```
module arbitrary (input clk, a, b, input [2:0] c,
                  output y, output reg z);

    and g1 (w1, a, c[2]);
    not g2 (nc, c[0]);
    xor g3 (w2, c[1], b);
    nor g4 (pz, w1, w2, nc);
    and g5 (y, w2, nc);

    always @ (posedge clk)
      z <= pz;

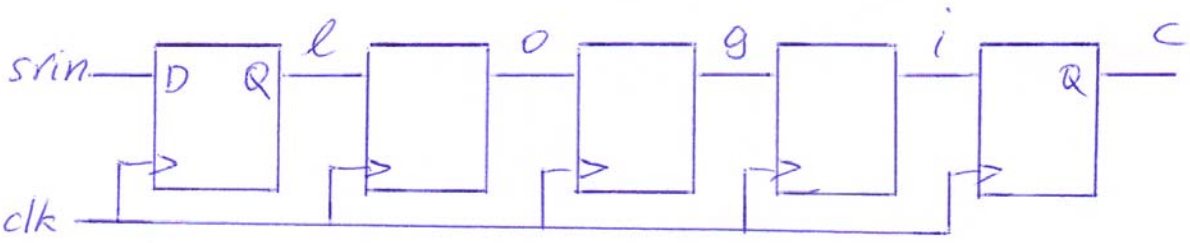
endmodule
```



Q2

What does the following code implement?

```
always @(posedge clk)
begin
    if (rst) begin
        c <= 1'b0; g <= 1'b0; i <= 1'b0; l <= 1'b0; o <= 1'b0;
    end else begin
        o <= l;
        c <= i;
        g <= o;
        l <= srin;
        i <= g;
    end
end
```

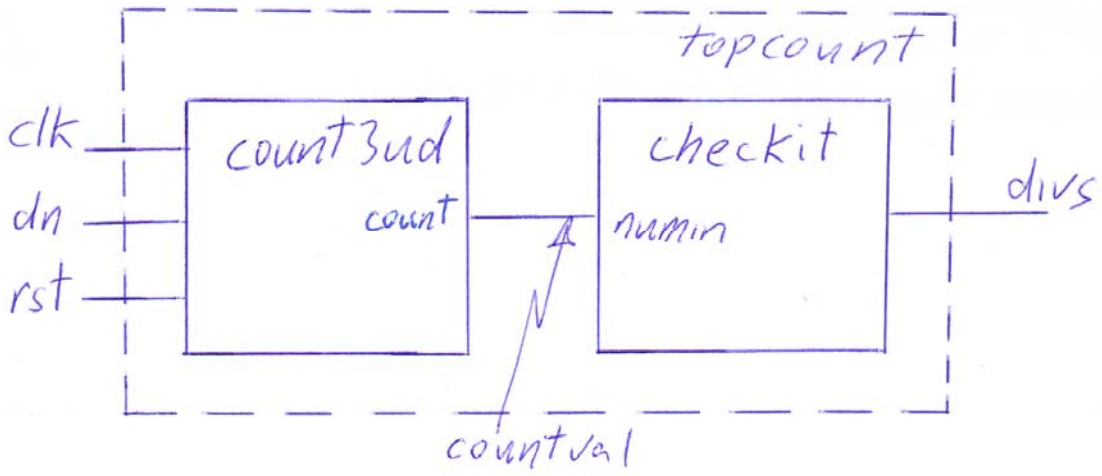


Q3 (a) 3-bit Up/Down counter

```
module count3ud (input clk, rst, dn, output reg [2:0] count)
    always @(posedge clk)
    begin
      if (rst)
        count <= 3'd0;
      else
        if (dn)
          count <= count - 1'b1;
        else
          count <= count + 1'b1;
    end
endmodule
```

The *checkit* module just implements some arbitrary function

```
module checkit (input [2:0] numin, output divs)
```



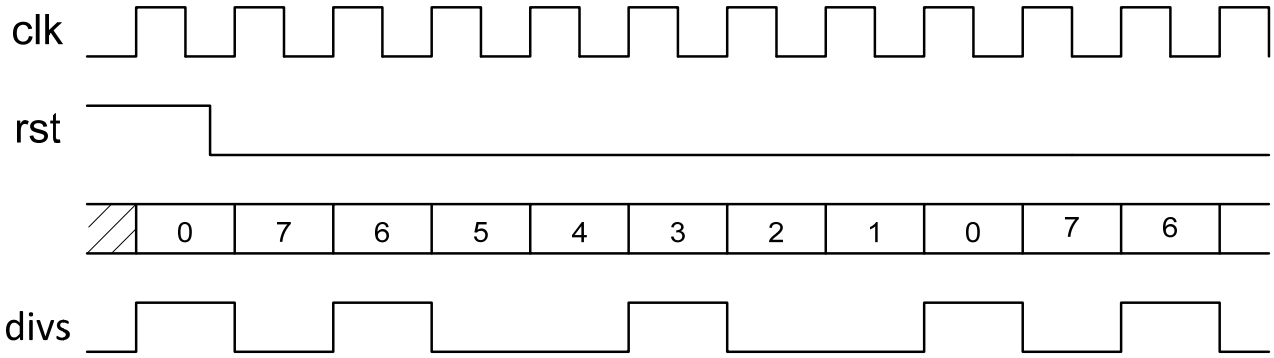

```
module topcount (input clk, rst, dn, output divs)
    wire [2:0] countval;
    count3ud mod1 (.clk(clk), .rst(rst), .dn(dn), .count(countval));
    checkit  mod2 (.numin(countval), .divs(divs))
endmodule
```

Q3 (b) Show timing diagram

Firstly, what does module checkit do? Lets draw a TT to find out.

numin[2:0]		divs
0	000	1
1	001	0
2	010	0
3	011	1
4	100	0
5	101	0
6	110	1
7	111	0

So we can generate a timing diagram for a down counter



Q3 (c) What does it do?

Checks for divisibility by 3. Also divides clock by a non-power-of-2 value.

Q4 What is Functional Verification and Timing Verification

Functional verification:

- Check the code describes the correct function
- Done at the design entry stage
- Iterative processes with design

Timing verification:

- Check the design meets timing requirements
- Done after place and route
- Can use the same testbench as functional, but use the post-PAR models from the tools